

Lean Management of Software Processes and Factories Using Business Process Modeling Techniques

Javier Berrocal, José García-Alonso, and Juan Manuel Murillo

Escuela Politécnica, University of Extremadura,
Av. Universidad S/N, 10071, Cáceres, Spain
{jberrolm, jgaralo, juanmamu}@unex.es

Abstract. The software industry is moving towards a software factory business model, usually involving several centres collaborating on company contracts. The expected benefits of using specialized teams at lower cost locations are increased productivity and reduced costs. The tasks of project and process management have as a consequence become more complex. Managing such large structures requires more collaboration in development processes to enable rapid reaction to project needs, and support for the variety of technologies, methods, and levels of quality required by the different projects. This situation demands new practices and management support tools. This paper presents Zentipede, a tool for software process management. Its focus is on lightening, or even automating, management tasks by using Business Process Management (BPM) techniques. The tool does not force any particular practice on a company, but encourages it to model the practices which will finally be automated. Also, it supports process-to-product traceability.

Keywords: Software Process and Project Management, Distributed Software Development, Software Process Quality, Process and Project KPI.

1 Introduction

In the last few years, software development companies have changed their business strategies, objectives, and processes to adapt to new customer demands and market trends.

One of the main changes has been the adoption of agile development processes. The essence of these processes is increased communication and collaboration among developers, fostering a self-organizing culture and encouraging development tailored to customer needs [1], [9].

Another change has been the evolution of these companies towards the software factory business model. They now site their development centres (factories) at lower cost locations near their zone of action (nearshore) or even far away (offshore) [10], [11]. These factories usually specialize in specific development activities or technologies, so that the implementation of a project is distributed among different factories according to their specialities and workload. Thus, although the software factory business model increases productivity and reduces development costs [12], it has a

negative impact on communication and task completion [19], [20]. So, its full benefits can only be obtained by deploying highly industrialized development processes and tools that stimulate collaboration, and by closely monitoring the workload on resources to ensure their rapid reallocation [17].

Finally, since quality certification has become one of the most important assets contributing to software companies' competitiveness [30], they are adopting models to manage the quality of their software processes [2] and even the entire life-cycle of the services they provide [3], [4]. Customers of IT products and services no longer make their project investment decisions solely on the basis of development costs and time. Today, the final quality of the product or of the process followed to develop it is equally or even more important. This trend is another key factor contributing to the increasing effort companies are putting into the optimization of management tasks.

Hence it is clear that companies' management practices need both to be streamlined and to increase in scope [8]. The use of software management tools can contribute to resolving this dilemma. Examples are the so-called software cockpits, also known as Software Project Control Centres (SPCC) [5] or Project Management Offices (PMO) [6]. They are specially designed to provide support for project managers' most important activities.

However, these tools usually do not cover all the areas and roles that are involved in a company's process management, resulting in managers using separate tools for different areas [14], [15]. This leads to duplication of effort and a loss of productivity because the same or similar data must be provided to each tool while guaranteeing the coherence of the overall system [13]. A further drawback is that these tools are not usually designed to be adapted to an organization's changing needs [7], and thus end up constituting a barrier to the evolution of its business strategies and the improvement of its processes.

Companies therefore require integrated tools which cover all the tasks and roles of the management of their software processes, projects, and factories [13], [14]. These tools should be capable of dealing with different processes and life-cycles, providing management with support for both software product development and IT services. They should also be able to adapt to any new strategies or processes that the company wants to implant. This paper presents Zentipede¹, a suite of integrated tools developed with the objective of meeting the above needs – lightening or even automating the widest possible range of management activities. It does not force any particular practice on management. Instead, it allows them to model their practices using Business Process Management Notation (BPMN). These practices are then automated by a Business Process Management System (BPMS) which delivers orders to the management tools. These tools support process, product, and process-to-product traceability.

The paper is organized as follows. Section 2 briefly describes some of the tools used to manage and control software development. Section 3 presents Zentipede, detailing its architecture and modules. Section 4 describes the results of using Zentipede. Finally, Section 5 presents some conclusions and outlines future work.

¹ Although the correct spelling is Centipede, a “Z” was substituted for the “C” when composing the image of the product to associate it with a sinuous centipede in the form of a Z.

2 Background and Related Work

Since management is one of the keys to the success of a software project, its optimization has long attracted the attention of companies and researchers. Some of these efforts have been devoted to producing configuration management [21] or documentation tools [16], provide support to developers and improving their collaboration and coordination.

Another trend has focused on the development of tools (SPCC [5] or PMO [6]) to provide greater control over software processes. Examples such as Rational Team Concert [23] or Artemis 7 [27] include features that facilitate the implementation of iterative and agile software processes. Other tools generate statistics about a project's status. An example is Specula [7] which includes a method to help project managers define metrics aligned with the project's objectives.

Since dealing with process improvement involves a major investment in quality assurance activities [29], many tools (such as SIMPLe [22] or MKS [28]) have been developed to help reduce this effort. These tools facilitate the quality assurance managers' tasks by implementing key good practices or generating the artefacts defined by their models.

Table 1. Functionalities covered by some tools

	Configuration management	Documentation management	Traceability	Users management	Soft. Process management	Soft. Process monitoring	Adaptation to B. Strategy	Management tasks lightened	Software Quality	Quality management
SVN	Yes	No	No	No	No	No	No	No	No	No
TWiki	No	Yes	NO	No	No	No	No	No	No	No
Rational Team C.	Yes	Yes	Process Product	Yes	Yes	Yes	No	No	No	No
Artemis 7	No	No	No	Yes	Yes	Yes	No	No	Yes	Yes
Specula	No	No	No	Yes	Yes	Yes	Yes	No	Yes	Yes
SIMPLe	No	No	No	Yes	No	No	No	No	Yes	NO
MKS	No	No	Process	Yes	Yes	Yes	No	No	Yes	Yes

Notwithstanding the utility of these tools in reducing the effort spent in each area, they still function independently, as is shown in Table 1. In particular, specific information has to be provided to each of them separately. In many cases, this information is similar or even the same, so that the result is a duplication of effort and loss of productivity [13] that could be avoided by using integrated tools. In addition, some management activities such as task allocation, project monitoring, and resource management are highly repetitive. Tools to automate as many of these management activities as possible would therefore also be greatly welcomed.

Finally, software companies and software processes are continuously evolving to cover new necessities, trends, and demands. Management tools should therefore also be capable of adapting to new strategies or processes [24]. Zentipede was born with

this motivation, as well as with the aim of providing centralized monitoring of process and project status. Also, it is an open-source tool.

3 The Zentipede Approach

This section introduces Zentipede. The origin of the suite was the perceived need for integrated management tools, capable of dealing with distributed factories and various process models, and of collecting and producing statistics about project status, factory workloads, resources productivity, the typical project Key Performance Indicators (KPIs), etc. The suite was designed to adapt to any company, as well as to support any need for evolution that the company might have. Figure 1 shows the Zentipede architecture.

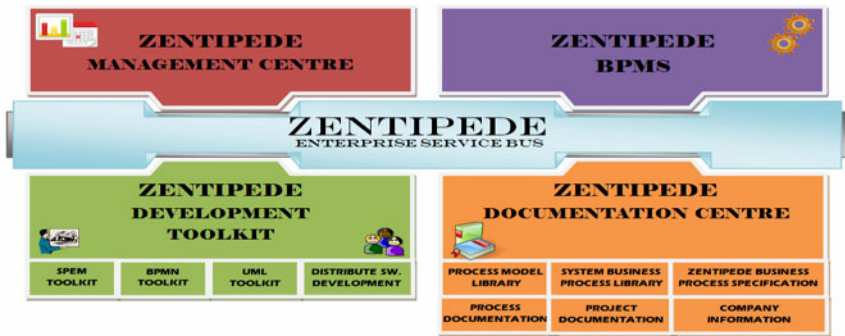


Fig. 1. Zentipede architecture

The Zentipede architecture is divided into three layers and four modules:

- *Process Layer*. The layer in charge of managing the processes and projects deployed by the company. Two modules are responsible for covering all management activities, collecting data, monitoring projects, and lightening or automating management tasks. They also implement features that allow the information stored to be browsed. This information can be browsed starting from different points of view.
- *Modeling and development layer*. The layer in charge of providing support for modeling and development tasks. The modules comprising this layer were specifically designed to provide support to distributed task groups. This layer also stores and synchronizes the company's knowledge databases.
- *Connection Layer*. The layer in charge of connecting and integrating (by means of an Enterprise Service Bus) the modules of the other two layers.

The main motivations and responsibilities of each module are the following:

- Software processes management and monitoring is ever more complex, requiring more tasks and more experienced managers. *Zentipede BPMS* has been developed with the objective of lightening or even automate some of these tasks. It supports

the specification of the company's software processes using Software Process Engineering Metamodel (SPEM) and BPMN. These processes can differ depending on the applicable quality level, the management workload allocated to the project, the technology to be used, and the client's requirements. Once modeled, they are executed by a BPMS whose results are delivered as orders to the Zentipede Management Centre.

- Project managers have to know the project status to make the right decisions. This requires keeping track of the tasks being undertaken, their data and results. *Zentipede Management Centre* has been developed to facilitate this activity. For all practical purposes it behaves as a process management tool. However, tasks can be inserted manually by users, or automatically by Zentipede BPMS as a result of the execution of a software process. This module collects information on the tasks being undertaken in order to monitor the status of each project, and to link the data so as to maintain process traceability. These data are used to extract different KPIs. Also, since all the tools are integrated, they are capable of reusing this information, thus avoiding the duplication of effort.
- Software development is more complex in distributed environments due to additional problems introduced by special communications and coordination needs. *Zentipede Development Toolkit* has been developed to minimize some of these problems. In addition to a BPMN Toolkit, to model business processes, and a UML Toolkit, to model software products, some Eclipse plug-ins have been developed for the support of distributed development. As does any configuration management tool, these plug-ins facilitate synchronization among developers. But they also store each change made to the source code, linking that change with the task causing it (which is registered in the Zentipede Management Centre). In this way, one has process-to-product traceability.
- Create the projects and processes documentation is a highly collaborative activity. It requires developers working together to continuously update their status. In distributed environments, this activity is even more complex because of the coordination problems, as those stated in [19]. *Zentipede Documentation Centre* has been developed to facilitate this activity. It allows developers, regardless their location, to work together to register all the company's knowledge used by Zentipede. Examples are the processes Zentipede is following, the business processes of the systems developed, and the different artefacts generated. This data allows the company to manage all of its knowledge, facilitating evolution.

In the following, each of Zentipede's modules will be described in detail, explaining its objectives, its functionalities, and the plug-ins or tools that it contains. In addition, an example will be presented of the integrated use of all these modules.

3.1 Zentipede's Main Workflow

Figure 2 shows the usual workflow followed when these tools are used in an integrated form. It needs to be borne in mind that, despite the simple way in which the steps are presented, Zentipede is a complex tool covering a wide range of tasks. Imagine a company that wants to start developing a project with Zentipede. The steps that would normally be taken using Zentipede are the following:

1. The software processes to follow in developing the project have to be defined. To this end, the processes are modeled (if they have not already been modeled) with *SPEM Toolkit*. The information generated is stored in the *Process Model Library* to be reused in future projects.
2. Once the software processes have been modeled, they are translated internally to BPMN using *BPMN Toolkit*.
3. The new project has to be added to the *Zentipede Management Centre*. As part of this action, additional information is recorded, such as the project leader, the technology, etc.
4. The project leader defines the development teams.
5. An instance of the software process followed is deployed in *Zentipede BPMS*. As a result, the process begins scheduling the first tasks for the development teams to do. Associated with these tasks is an estimate of the effort required. The tasks are generated semi-automatically by *Zentipede BPMS*, but the effort estimate has to be provided by those responsible for each task.
6. Each team leader reserves for each of his or her team's members the number of hours to be dedicated to the project (during the next iteration, sprint, week, or evaluable time period). Team leaders can divide the tasks into subtasks and allocate them to resources through the *Zentipede Management Centre*.
7. The developers perform their assigned tasks using the tools provided in *Zentipede Development Centre*, through which they can also report results to *Zentipede Management Centre*.
8. Once results are obtained, they are interpreted by *Zentipede BPMS* so as to update the software process consistent with the project's status.

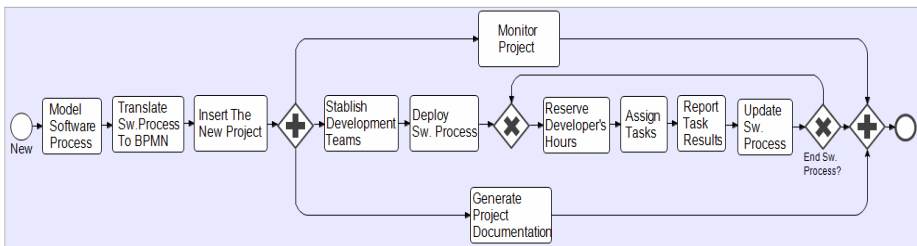


Fig. 2. Basic business processes of Zentipede

Throughout the project, *Zentipede Management Centre* generates metrics to evaluate the status of the process, the project, and the company objectives.

3.2 Zentipede BPMS

Zentipede BPMS is the module that executes software processes, lightening or even automating a wide range of management activities. The specific objectives which guided its development were:

- To supervise and advise on the work to be done to complete the software process being followed.

- To monitor the status of each process and project.
- To automate as many management activities as possible.

In order to fulfil these objectives, an engine, termed a BPMS, was needed to execute business processes. A BPMS is an engine that executes business processes, normally modeled with BPMN or BPEL, to orchestrate the tasks and interactions that are defined in them. In this case, this engine had to execute the software process followed in each project.

To develop this engine, we evaluated existing BPMSs (Apache ODE [25] and Intalio BPMS [26]). We decided to develop the Zentipede BPMS by adding features to Apache ODE to support the characteristics of software process execution and software development. Among these features, new attributes were added to the BPMN specification to include specific information on the software process, such as:

- Artefacts or models that should be generated as a result of each task.
- Modules of Zentipede that should be used to complete each task.
- Automatic or manual activities. Automatic activities are those that can be completed by Zentipede without user intervention. Manual activities are those that at least one user has to work on.

BPMN Core Elements	Attributes	BPMN Core Elements	Attributes
Events Start  Intermediate  End 	Users Roles Guidance	Swimlanes Pool  Lanes (within a Pool) 	Users Roles Guidance
Activities Task  Sub-Process 	Users Roles Tools Auto/manual	Gateways   	Users Roles Auto/Manual Guidance
Connecting Object 	Condition	Artifacts Data Object 	Artifacts: IN /OUT Mandatory / Optional

Fig. 3. BPMN elements and the attributes used in the BPMS

Figure 3 shows some of the attributes added to the BPMN elements. Most were identified by observing the information already modeled with SPEM, and extracting those attributes that were useful for the execution of the software process.

The following is a brief description of the main steps in the Zentipede BPMS workflow:

- When a decision has to be made as to what software process to follow, the project leader either models a new one or chooses one from among those stored in Zentipede Documentation Centre.
- A new instance of the chosen process is deployed in the BPMS.
- Using that instance, the project is monitored and, as it advances, the tasks defined in the process are scheduled.

- Automatic tasks are completed by the BPMS, reusing previously stored information (for example to update some part of the documentation, it can reuse the information already inserted in other tasks or in the artefacts that have been generated).
- Manual tasks are automatically added to Zentipede Management Centre, and the roles responsible for each task are notified that they have to be assigned to developers.

However, although some projects may initially follow the same software process, as each project advances the processes will eventually evolve differently. Therefore, software processes must be flexible enough to adapt to the status of each project.

To provide this flexibility, software processes can be modeled such that they comprise many sub-processes. Then Zentipede BPMS will execute each sub-process depending on the artefacts generated and the status of the corresponding project. For example, how some sub-processes are executed will depend on the elements and interactions between the product's business processes, the number of use cases defined in the use case diagram, or the packages and interactions between classes of the class diagram. Figure 4 shows a sub-process for a use-case driven development modeled by OpenUP, in which the number of instances created for this sub-process depends on the number of use cases modeled in the use case diagram.

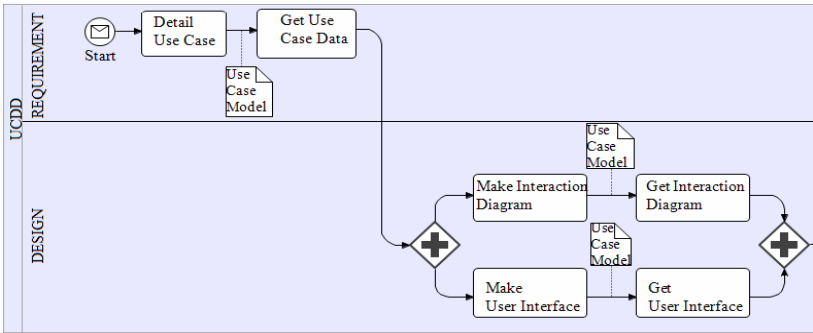


Fig. 4. A sub-process used for use-case driven development

Thus the activities defined in a software process are monitored by executing the process, leading to automating, or at least lightening, many management activities such as task assignment or progress logging. Also, the process can be modeled and executed in a way that endows it with the flexibility needed for its continuing adaptation to the progress of the project.

3.3 Zentipede Management Centre

Zentipede Management Centre is the most important module of Zentipede. It covers the basic functionality for the management of projects and processes. Its main characteristics are:

- Supporting software development distributed among different software factories.
- Providing support for the management activities of all of a software company's roles.
- Collecting data on processes, projects, and factories while they are being managed, and enabling this information to be browsed from different points of view.
- Generating statistics for project, process, and factory monitoring.

To fulfil these objectives, Zentipede Management Centre was designed as a web application with a three-tier architecture using the latest technologies. It can be deployed in the context of any organization, and accessed by any of the personnel, regardless of location.

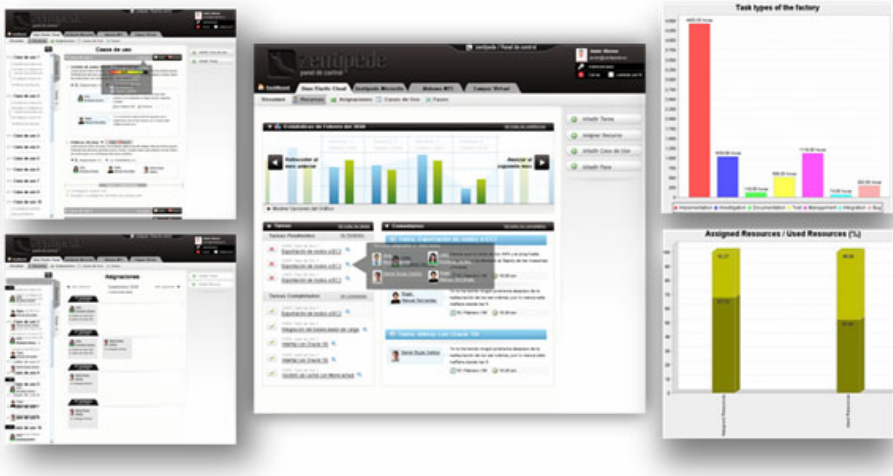


Fig. 5. Zentipede Management Centre: (a) Snapshot; (b) KPIs

This module includes the definitions of the software company's principal roles, with each role assigned a series of responsibilities and obligations. The main functionalities implemented to cover these obligations are detailed in the following paragraphs (see also Figure 5a), but it should be borne in mind that the roles and their obligations are highly flexible, so that each company can define new roles, and assign obligations that are in accordance with the company's organizational structure.

- *Company managers.* Their activities may include managing factories (adding new factories and updating their information), projects (adding projects and assigning project leaders), or the company overall (defining policies and rules).
- *Factory managers.* They can control information on personnel, tasks performed within the factory, projects in which the factory is involved, etc.
- *Project leaders.* They can decide which software process is to be followed, divide the project into iterations, and decide what requirements are to be addressed in each iteration. In addition, they can constitute and supervise development teams.
- *Team leaders.* They control the developers comprising their team, detailing information and reserving the number of hours that each developer is to dedicate to the

project during the next iteration. Unreserved hours denote the developer's availability to collaborate in other teams or projects. Zentipede supports the possibility that a resource may have hours reserved that, for some reason or other, are eventually not used. Team leaders can also divide requirements into more specific tasks, estimate the effort required to complete them, and assign tasks to developers.

- *Developers*. They can see what tasks they have been assigned, and the details, estimates, and documents to generate for each of those tasks. Additionally, they can report the actual effort devoted to each task, and the documents that were generated.
- *Quality managers*. They can automate the generation of reports on the status of each project or factory.

Zentipede Management Centre continuously gathers large quantities of data – the phases or iterations of each project, the requirements covered in each iteration, the tasks into which the requirements were divided, the developers who worked on each task, the estimated effort for each task and the actual effort finally to it, the time reserved for each resource and the actual time used, etc. The collection of this data is transparent to the users. The information is stored in Zentipede Documentation Centre for subsequent reuse by Zentipede Management Centre. This yields a twofold benefit – first, it reduces the new information required by other tasks, and second, it enables KPIs to be automatically generated to help users monitor and evaluate the areas they are responsible for (Figure 3b).

- *Company managers* can see KPIs on costs, profits, company efficiency, and the average level of occupation of each factory.
- *Factory managers* can evaluate the productivity of the factory overall or of each employee in particular, and view the contribution of each project to the factory's workload.
- *Project leaders* can view statistics about the progress of a project, the averages of resources assigned and actually used, and gaps in the estimates.
- *Team leaders* can view metrics about the tasks undertaken or the work done by each resource.
- *Quality managers* can set up controls to evaluate the quality of processes, the efficiency of developers, or bugs in the products.

Each user can also adapt these indices to their own needs. To that end, the indices are parametrized, allowing such changes as the level of detail, the form in which the information is displayed, and the source of the data. Additionally, new indices can be inserted based on the data collected.

Finally, as the data is collected, it is immediately linked to the previously stored data to provide process traceability, allowing users to browse through the information from different points of view. For example, starting with a given project, they can view its objectives and iterations, and inquire deeper into the requirements addressed in each iteration, the tasks proposed to satisfy those requirements, the resources dedicated to each task, and the effort and dates spent on carrying it out. Starting with a given resource, they can view the projects on which they collaborated, the goals that were attained, the tasks carried out and the time spent on each, etc. Or, starting with a date, they can see the tasks and requirements carried out on or up to that date, the resources involved in them and their workload, or the status of each project.

3.4 Zentipede Development Toolkit

Zentipede Development Toolkit is a suite of tools that facilitates the creation of artefacts with information about the company or project results. The specific objectives which guided its development were:

- To provide tools to mitigate the handicaps of distributed software development.
- To facilitate the creation of artefacts with information about the company or projects.
- To collect information about the work done by each developer.

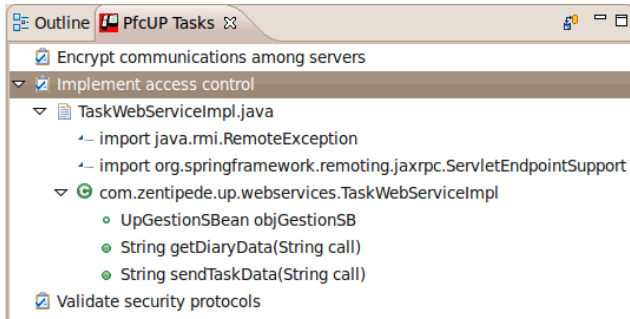


Fig. 6. Zentipede Development Toolkit

To fulfil these objectives, this module contains four subsets of tools:

- *SPEM Toolkit* is an Eclipse plug-in that allows company managers or quality managers to model the software processes that can be followed. Thus, the processes are defined and made accessible so that any user can see what they are to do in each task, why, and how to do it.
- *BPMN Toolkit* is an Eclipse plug-in that allows users to model business processes with BPMN. This tool can be used to model the business processes of the products that are being developed (with these models then forming part of the project's documentation), or to translate the software processes defined with SPEM into BPMN (these processes can then be executed by Zentipede BPMS).
- *UML Toolkit*. This subset includes Eclipse plug-in tools that facilitate the UML modeling of the artefacts required over a project's life-cycle. Developers can then access these plug-ins without having to change IDE.
- *Distributed Software Development Toolkit*. First, this subset provides tools to help developers communicate with each other. These tools are Eclipse plug-ins that allow developers to describe the doubts or problems they encounter in their assigned tasks. These doubts are automatically notified to all the personnel subscribed to the task (project leader, team leaders, and other developers). For all practical purposes, these plug-ins behave as communication tools, but they also store the communications and link them to the tasks that gave rise to them. And second, this subset includes another plug-in which facilitates developers' management activities (Figure 6). This plug-in shows each developer the tasks assigned to them in their working environment, and allows them to specify further information on each task.

It also automatically collects such information as the time spent on each task and each change made to the source code (such as which packages, classes, or lines have been modified). As this information is captured, it is immediately linked to the task that gave rise to it. With the information collected with this subset's plugins, the process traceability already supported by Zentipede Management Centre is enhanced with process-to-product traceability. This feature facilitates reverse engineering, allowing users to browse the information by starting with a given project, requirement, task, or resource in order to view the corresponding changes made to the source code or the doubts and problems that were encountered, and *vice versa*. It also reduces the impact of resource mobility and task reassignment by providing information on the changes already implemented in completing each task.

Finally, all the information captured or defined with these tools is stored in the module Zentipede Documentation Centre for reuse by other tools.

3.5 Zentipede Documentation Centre

Zentipede Documentation Centre is the module responsible for storing and maintaining the consistency of all the project and process data and company information. The specific objectives which guided its development were:

- To maintain libraries in which to store all the information collected.
- To manage all the software company's information and knowledge.

To fulfil these objectives, six integrated repositories were set up. They were assigned the following responsibilities:

- *Process Model Library*. This library stores the software processes that can be followed by project leaders. When a new project is started, the project leader selects the process to follow according to the project's needs and objectives, and this is executed in Zentipede BPMS to guide the management activities.
- *System Business Process Library*. This library stores the business processes of the products under development. When a new project is started, the processes can be modeled as part of the product's documentation, and as the first step to adapt the software process to the characteristics of the project.
- *Zentipede Business Process Specification*. This repository stores business processes defining the interactions between the tools of Zentipede.
- *Project Documentation*. This repository stores all the artefacts generated as a result of software process activities (use case diagram, class model, activity diagrams, etc.).
- *Process Documentation*. This repository stores information on the instances of the software processes being followed in each project. The information is stored as phases or iterations of each process, the requirements covered, the developers who worked on each requirement or task, the estimated effort needed by each task and the actual effort devoted to it, etc.
- *Company Information*. This repository stores information about the company's objectives and rules, such as business rules, objectives, or good practices.

These repositories store and maintain all the information on the processes, projects, and factories. Because all the modules are perfectly integrated, this information is used by the other modules to monitor each project or process's status, to generate indices, to lighten or automate certain tasks, and to maintain process and process-to-product traceability.

4 Validation and Evaluation of Zentipede

This suite of tools has already been tested in use in some software companies. The testing period lasted more than seven months, in which time more than ten projects were managed. Most of them were distributed over various locations, including Cáceres, Madrid, and Barcelona.

At the end of the testing period, members of those companies reported on its usefulness in managing their projects and factories. Some of the most interesting benefits that they indicated were:

- The capacity to generate indices to evaluate compliance with the company's objectives and rules.
- Constant evaluation of projects and processes, with rapid reaction to any deviations detected.
- Improvement of work coordination, including better coordination among factories and communication among distributed workers.
- Better use of manpower and increased productivity (some reports described a 25% to 50% increase in the use of manpower) due to the facility of monitoring each developer's workload. In this sense, some of the managers pointed to the value of the feature of reserving the developers' hours, and the statistics comparing the reserved hours and the actual hours used, since they had detected that some team leaders were reserving more hours than necessary to simulate that their project was advancing faster than was really the case.
- A reduction in the impact of incorporating new developers, of allocating tasks that had already started to developers, or of solving bugs, due to the use of process and process-to-product traceability.

Although, these tests were very promising, this tool is not yet in the production use because we are adding new functionalities. Once these functionalities are finished, a more scientific evaluation of the tool will be done and, then, it will be used in production.

5 Conclusions

This paper has presented Zentipede, a tool designed to cover all areas of project, process, and software factory management. For the coverage of each area, four modules were developed comprising many perfectly integrated and coordinated tools. One of the benefits of this integration is that the information inserted in one module is reused by the others. For example, the information captured during the process management can be used to create the documentation or to automatically generate quality indices. This contrast with the tools listed in Table 1, in which the similar or even the

same information has to be provided to different tools for different purposes. Therefore, this integration increases the effectiveness of each module and the data consistency. In addition to this benefit, together with the process and product traceability, already provided by other tools, the modules integration allows information to be captured to achieve process-to-product traceability.

As it has been seen throughout the paper, methods and tools facilitating the software project management are required. Zentipede incorporates a module that executes software processes using BPM techniques to lighten or even automate a wide range of the management tasks. This feature has not yet been covered by the tools evaluated in Table 1.

Finally, Zentipede was designed to be adaptable to the characteristics of any environment or company, taking into account their present or planned future strategies and processes.

Acknowledgments. This work has been funded by PDT08A034, TIN2008-02985, GRU09137, PRE09156 and Fundación Valhondo Calaff.

References

1. Agile Manifesto, <http://www.agilemanifesto.org/>
2. CMMI, <http://www.sei.cmu.edu/cmmi/>
3. ITIL, <http://www.itil-officialsite.com/home/home.asp>
4. CMMI for Services, <http://www.sei.cmu.edu/cmmi/tools/svc/>
5. Münch, J., Heidrich, J.: Software Project Control Centers: Concepts and Approaches. *J. Syst. and Soft.* 70(1), 3–19 (2003)
6. Project Management Institute: A Guide to the Project Management Body of Knowledge (PMBOK Guide). Project Management Institute, Pennsylvania (2000)
7. Heidrich, J., Münch, J.: Goal-Oriented Setup and Usage of Custom-Tailored Software Cockpits. In: Jedlitschka, A., Salo, O. (eds.) PROFES 2008. LNCS, vol. 5089, pp. 4–18. Springer, Heidelberg (2008)
8. Jennings, T.: Planning for value. Technical report, Butler Direct Limited (2005)
9. Larman, C.: Agile & Iterative development: a manager's Guide. Addison Wesley, Boston (2005)
10. Johnson, J.R.: The Software Factory: Managing Software Development and Maintenance. John Wiley & Sons, Portland (1991)
11. Greenfield, J., Short, K., Cook, S., Kent, S.: Software Factories: Assembling Applications with Patterns, Models, Frameworks, and Tools. Wiley, Indianapolis (2004)
12. Sengupta, B., Chandra, S., Sinha, V.: A research agenda for distributed software development. In: 28th International Conference on Software Engineering, pp. 731–740. ACM, New York (2006)
13. Sinha, V., Sengupta, B., Gosal, S.: An adaptive tool integration framework to enable coordination in distributed software development. In: 2nd International Conference on Global Software Engineering, pp. 151–155. IEEE Comp. Society, Washington (2007)
14. Krishnamurthy, V.: Benefits of Tool Integration In Distributed Agile Development. *AgileJournal* 1 (2006)
15. Maurer, F., Dellen, B., Bendeck, F., Goldmann, S., Holz, H., Kötting, B., Schaaf, M.: Merging Project planning and Web enabled dynamic workflow technologies. *IEEE Internet Computing* 4, 65–74 (2000)

16. TWiki, <http://twiki.org/>
17. Ambler, S.W., Nizami, K.: Agile Strategies for Geographically Distributed Quality Management. *AgileJournal* 2 (2007)
18. Prikladnicki, R., Audy, J.L.N., Damian, D., de Oliveira, T.C.: Distributed Software Development: Practices and challenges in different business strategies of offshoring and onshoring. In: 2nd International Conference on Global Software Engineering, pp. 262–274. IEEE Computer Society, Washington (2007)
19. Nguyen, T., Wolf, T., Damian, D.: Global software development and delay: Does distance still matter? In: 3rd International Conference on Global Software Engineering, pp. 45–58. IEEE Computer Society, Washington (2008)
20. Grinter, R.E., Herbsleb, J.D., Perry, D.E.: The Geography of Coordination: Dealing with Distance in R&D Work. In: ACM SIGGROUP conference on Supporting group work, pp. 306–315. ACM, New York (1999)
21. Subversion, <http://subversion.tigris.org/>
22. SIMPLE, <http://alturasoluciones.com>
23. Rational Team Concert, <http://www-01.ibm.com/software/awdtools/rtc/>
24. Selby, R.W.: Analytics-Driven Dashboards Enable Leading Indicators for Requirements and Designs of Large-Scale Systems. *IEEE Software* 26, 41–49 (2009)
25. Apache ODE, <http://ode.apache.org/>
26. Intalio BPMS, <http://community.intalio.com/>
27. Artemis 7, <http://us.aisc.com>
28. MKS Integrity Suite, <http://www.mks.com/>
29. Kelly, G.: Barriers to adoption of the CMMI process model in small settings. In: 1st Workshop for Process Improvement in Small Settings, pp. 36–40. SEI, Pittsburgh (2006)
30. Barnes, F.: Good Business Sense Is the Key to Confronting ISO 9000. *Review of Business* (2000)